

THE CONSTRUCT GAME SERVICES

View online: <https://www.construct.net/en/game-services/manuals/game-services>

Overview

Welcome! The Construct Game Services (CGS) are a suite of services designed to help you add advanced and more interactive features into your games without needing your own servers or backend. The services are designed to be flexible and modular, so you can use as many or as few as you need.

Implementation of the CGS into your game/application can significantly increase player retention and stickiness as well as providing valuable insights into how your game/application is being used.

These services are platform agnostic - you don't have to just use them in Construct 3, you can use them with any other game engine. You can even use them for non-game applications, for example the Leaderboard Service could be used in business sales team to track employee performance. You can be as creative as you want and if you're using it for something outside the box we'd love to hear/see what you're building!

CGS are currently in Beta. You may experience service interruption and some outages as we learn about how CGS is being used and adjust our infrastructure and address bugs as they come up.

If you find a bug or issue, please submit it to the official bug tracker at <https://github.com/Scirra/Construct.net-website-bugs>.

Authentication Service

Allow players to sign in to your game/application with `UsernamePassword`, `Apple`, `BattleNet`, `Discord`, `Facebook`, `Github`, `Google`, `ItchIO`, `Microsoft`, `Patreon`, `Reddit`, `Steam`, `X`, `Yandex`.

Once a player is signed in, they can also:

- Set/update their player name (displayed to other users)
- Set, delete or update their player avatar
- Link their account to other login providers allowing for multiple sign in options
- Set a username/password to sign in with
- Delete their account

This service is the core of the CGS - the players ID is used to identify them and allow them to interact across all other services.

Broadcasts Service

Allows you to create **Broadcast Channels**, and from these channels create **Broadcast Messages** that are accessible to all players in your game/application. For example, you might create a channel "New Features" and publish messages into this channel describing new features you have introduced when you update your game.

Once your message is published to your channel, it can be requested from your game/application auto-translated into the following languages: **AR** , **BG** , **CS** , **DA** , **DE** , **EL** , **EN-GB** , **EN-US** , **ES** , **ES-419** , **ET** , **FI** , **FR** , **HE** , **HU** , **ID** , **IT** , **JA** , **KO** , **LT** , **LV** , **NB** , **NL** , **PL** , **PT-BR** , **PT-PT** , **RO** , **RU** , **SK** , **SL** , **SV** , **TH** , **TR** , **UK** , **VI** , **ZH-HANS** , **ZH-HANT** .

You can also see how many players have read each message you publish. If setup, you can also allow players to rate messages that are published in multiple dimensions giving you valuable feedback and increasing engagement.

Cloud Save Service

Allow players to save blobs of data into the cloud for retrieval at a later date. The most useful and simple application for this is allowing players to save their game progress. You can also save a picture alongside a cloud save to represent the data more visually - for example a screenshot of the game where the save game was created.

You can also create public **Game Buckets** that players can save data into - for example if your game allows you to create custom levels, you could create a public bucket for these levels and allow players to share their creations with other players. These data blobs can also be rated with multiple dimensions, and returned to players sorted by the ratings.

Leaderboards Service

Create multiple leaderboards for your game/application. Unlike other services, CGS leaderboards automatically track scores by country allowing you to see how players from different countries are performing for specific leaderboards.

CGS leaderboards boast a large array of advanced features built in:

- Format scores for all types of values, time, money, points etc.
- Group players into Tiers/Divisions
- Assign players to teams and customise how teams are ranked
- Show daily, weekly, monthly or annual leaderboards
- Track players historic performance showing how their rank is improving over time
- Shadow ban players to prevent cheaters from spoiling the experience of legitimate players

- Set up thresholds that automatically reject scores if they fall outside of expected ranges
- Store additional values with scores to provide extra context when viewing leaderboards (for example, how many gold coins were collected)

Translations Service

Translate various translatable properties from other services into multiple languages. Supported target languages for translations are **AR, BG, CS, DA, DE, EL, EN-GB, EN-US, ES, ES-419, ET, FI, FR, HE, HU, ID, IT, JA, KO, LT, LV, NB, NL, PL, PT-BR, PT-PT, RO, RU, SK, SL, SV, TH, TR, UK, VI, ZH-HANS** and **ZH-HANT**.

Construct 3 Plugin

It's easy to interact with the leaderboard service directly within Construct 3 using the official [Construct Services Plugin](#).

Construct Arcade

If you use the [Construct Services Plugin](#) in your Construct 3 project, the plugin has methods to interact with the Construct Arcade Leaderboard. When you upload your game to the [Construct Arcade](#) a leaderboard will be created and configured automatically for you. Please note, for scores to be submitted to the Construct Arcade users must be logged in on Construct.net, guest scores are not permitted.

.NET Class Library

If you use .NET, you can install the [Construct Services class library](#) to use on your website or application.

The Construct.net website itself uses this library to run the [Construct Arcade](#) leaderboards, so will be well maintained and new features added as and when they are released.

Installation

Using the [.NET Core command-line interface \(CLI\) tools](#):

```
dotnet add package ConstructServices
```

Using the [NuGet Command Line Interface \(CLI\)](#):

```
nuget install ConstructServices
```

Using the [Package Manager Console](#):

```
Install-Package ConstructServices
```

Future Services

This is by no means a roadmap, but some of the services we are strongly considering developing for future releases. If you have a suggestion, we'd love to hear them!

- Achievements - allow players to progress towards awards or assign awards based on triggers.
- Polls - run polls players can vote on.
- Analytics - see how many players reach certain points in your game to help identify/address parts of the game that require more attention.
- Feedback - allow players to provide feedback for your game
- Chatrooms - create chat functionality in your games
- Inventories - define multiuser or consumable items and maintain player inventories in your game
- In Game Currency - allow players to earn in game currency and spend the currency on items in your game